



Week 1 - Generative AI use cases, project lifecycle, and model pre-training

Introduction to Week 1

- Deep Dive on  Transformer Architecture. Attention is All You Need paper introduced in 2017. Why did transformer architecture take off?
- Generative AI project lifecycle - walks through all of the stages and decisions you need to make while developing  Generative AI applications
 - Which open-source models to use?
 - How do we decide whether to fine-tune?
 - What is the size of the model to be used? - Single task models may only require small models



Generative AI

Machine that is capable of creating content that mimics human ability. Generative AI is a subset of traditional Machine Learning. And the machine learning models that underpin generative AI have learned these abilities by finding statistical patterns in massive datasets of content that was originally created by humans.



Large Language Models

- Trained over data from the internet on billions of words
- Some of the foundational or base models include
 - GPT
 - FLAN-T5
 - BERT
 - LLaMa

- BLOOM
- PaLM
- Models can be differentiated based on the number of parameters. The larger the model, the more memory it has, and can perform more complex tasks. The subjective understanding of the language for the model also increases
 - Smaller models can be fine-tuned to perform on small and very focused tasks
- Will use the open-source model FLAN-T5 to carry out language tasks
- Generative AI applicable for language, images, videos and audio, but focus only on Natural Language use cases in this course
- Terminologies
 - **Prompt:** Human Input to the model to perform a task
 - **Context window:** The space or the memory available to the prompt. Typically a few 1000 words, but differs from model to model
 - **Completion:** The output of the model
 - **Inference:** The act of using the model to generate text

LLM Use Cases and Tasks

Predicting the next word is the basis of all LLM output

- Essay
- Summarize text
- Machine Translation
- Generate code from Language
- Information retrieval - named entity extraction
- Augmenting LLMs by interacting with external APIs/Databases - providing information that is not available while pre-training and empowering model to interact with the real world

Researchers, over the years, have figured out that, the subjective understanding of the language the model increases as the number of

parameter increases. This understanding is what process and solves the task that the user has prompted

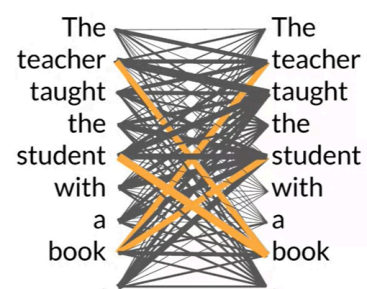
Text Generation before Transformers

- RNNs - while powerful for their time but limited by their computing and memory requirements needed to perform well at generative tasks
- Even after scaling, an architecture was needed to look beyond previous words and understand language basis the context of the entire sentence, paragraph, or even entire document
- 🧠 Transformer Architecture - Attention is All You Need was published in 2017 by Google and the University of Toronto
 - Allowed scaling efficiently
 - Parallel processing of input data - making use of larger training datasets
 - Able to learn to pay attention to the meaning of the words it is processing

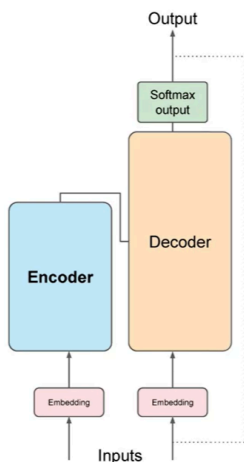
🧠 Transformer Architecture

The power of the transformer architecture lies in its ability to learn and understand the meaning of all words in a sentence by learning the mapping between every other word in the sentence. This is achieved by learning attention weights during training. We can use an **Attention Map** can be used to illustrate attention between each word and every other word.

In this attention map, the word book is paying attention to the words "teacher" and "student". This is called Self Attention, and the ability to learn attention in this way significantly improves the model's ability to encode language



How does the transformer model work?



At a high level, the transformer architecture is split into two distinct parts.

- The encoder, and
- the decoder

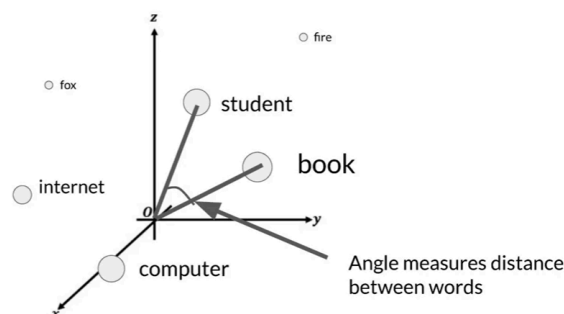
These components work are similar and work in conjunction with each other

Inputs

The inputs are passed through a Tokenizer that converts whole or part of text to numbers to be processed as input. Tokenizer can tokenize each unique word or parts of words. Once selected, must use the same tokenizer for training and inference.

Embeddings

- The inputs are then passed on to an Embedding Layer - a multi-dimensional vector space, that is trainable.
- Each token ID is matched to a multi-dimensional vector and the intuition is that these vectors learn to encode the meaning and context of individual tokens in input sequences.
- The vector size in the original transformer paper is 512.
 - For instance, in a 3-dimensional vector space, you can represent words like in the image below and measure the distance between them using the angle



Positional Encoding

Since the input processing is in parallel, you need positional encoding to preserve the word order in a sentence. The vector resulting from the sum of token embeddings and positional embeddings are passed on to the Self Attention layer.

Self Attention

Here the model analyses the relationships between the tokens in the input sequences. This allows the model to attend to different parts of the input sequences to better capture the contextual dependencies between the words. The self-attention weights identify the relationship between words and every other word.

This happens multiple times resulting in...

Multi-Head Attention

This means that multiple sets of Self Attention weights are learned in parallel - independently of each other. The number of attention heads varies from model to model (12 - 100 layers are common). **The intuition is that each self-attention head will learn a different aspect of a language**

- For example, a single head can focus on named entities
- Another head can focus only on the activities in the sentence

What the heads will learn is not dictated at the beginning, instead the weights are randomly initialized, and given sufficient time and training data, each head will learn different aspect. The interpretation of attention maps can also vary.

The outputs are then processed through

Fully Connected Feed Forward Network

The output of this layer is a vector of logits proportional to the probability score for each and every token in the tokenizer dictionary. These logits are passed through the final...

Softmax Layer

This outputs the probability of every single word in the vocabulary

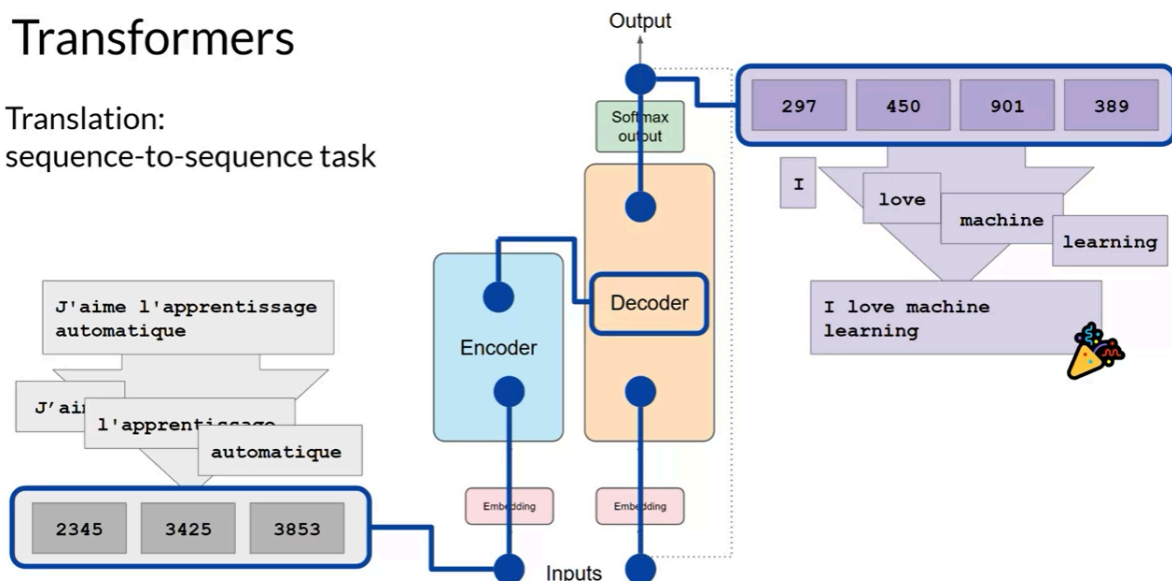
Example: Prediction through a transformer network

<https://www.coursera.org/learn/generative-ai-with-llms/lecture/R0xbD/generating-text-with-transformers>

This is an example of sequence to sequence task (which was the original objective of the transformer architecture) of Machine Translation - Convert French to English

Transformers

Translation:
sequence-to-sequence task



1. Tokenize the French sentence using the same Tokenizer used to train the network
2. These tokens are added as inputs to the encoder side of the network
3. They pass through the embedding layer and are fed to the Multi-Head Attention layer of the encoder
4. The outputs are fed to a feed-forward network as an output to the encoder. The encoder output encodes the structure and meaning of the input sentences
5. This representation is inserted in the middle of the decoder Self Attention mechanisms
6. A <SOS> start of sequence token is added as input to the decoder
7. This triggers the **decoder to predict the next token using the contextual understanding from the encoder.**
8. The output of the decoder passed through the decoder feed-forward network into the final Softmax output layer, which then generates the first token
9. Continue the loop by passing the output token back to the input of decoder to trigger the generation of the next token until the generation of <EOS> -

end of sequence token

10. Detokenize final sequence of tokens to words

Variations of Transformer models

The encoder and decoder parts can be split for variations. For example

- Encoder only models
 - used for sequence-to-sequence modeling where input and output sequences have the same lengths. This use is less common these days
 - Can perform a classification task like Sentiment Analysis
 - BERT is an example of an encoder-only model
- Encoder-Decoder models
 - Perform well on sequence-to-sequence tasks (e.g. Machine translation) where input and output sequence lengths can be different.
 - BARD and T5 are some examples
- Decoder only models
 - Include the GPT family of models
 - BLOOM, LLaMa, Jurassic

Prompting and Prompt Engineering

- Terminologies
 - **Prompt:** Human Input to the model to perform a task
 - **Context window:** The space or the memory available to the prompt. Typically a few 1000 words, but differs from model to model
 - **Completion:** The output of the model
 - **Inference:** The act of using the model to generate text
- Developing and improving a prompt is known as Prompt Engineering
- Providing examples inside the prompt is called In-context Learning

In-context Learning

- Helps model learn the task at hand

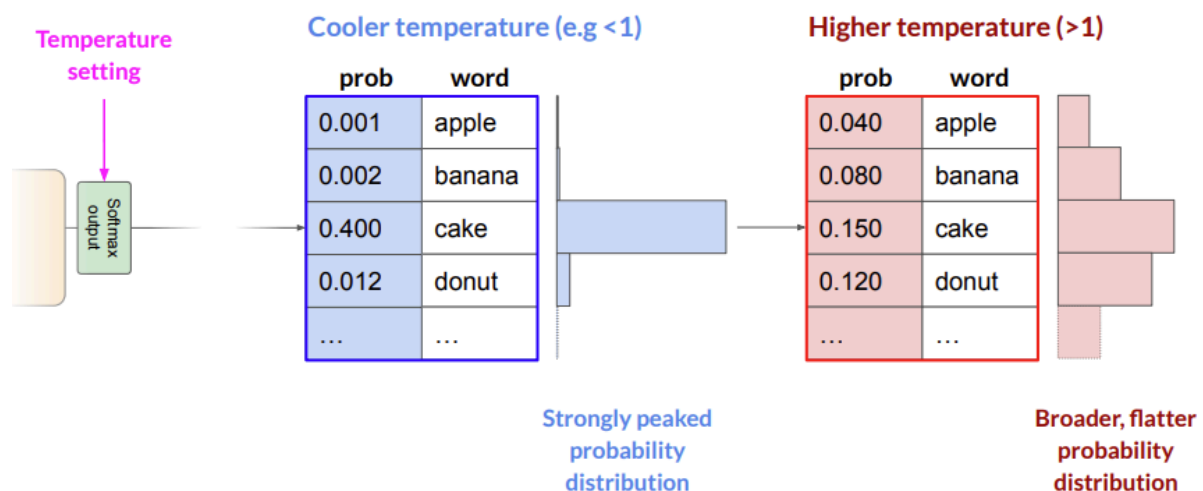
- Larger models are good at understanding the task and generating the expected output, smaller models can benefit from in-context learning by including one or more examples, as long as the examples fit the context window
- **Zero-shot inferencing**
 - Including the input in the prompt - large models can predict correctly, small models however can fail
- **One-shot inferencing**
 - Including one example in the prompt
- **Few-shot inference**
 - Even smaller model failing after including an example. Share more examples to perform better

Generative Configuration

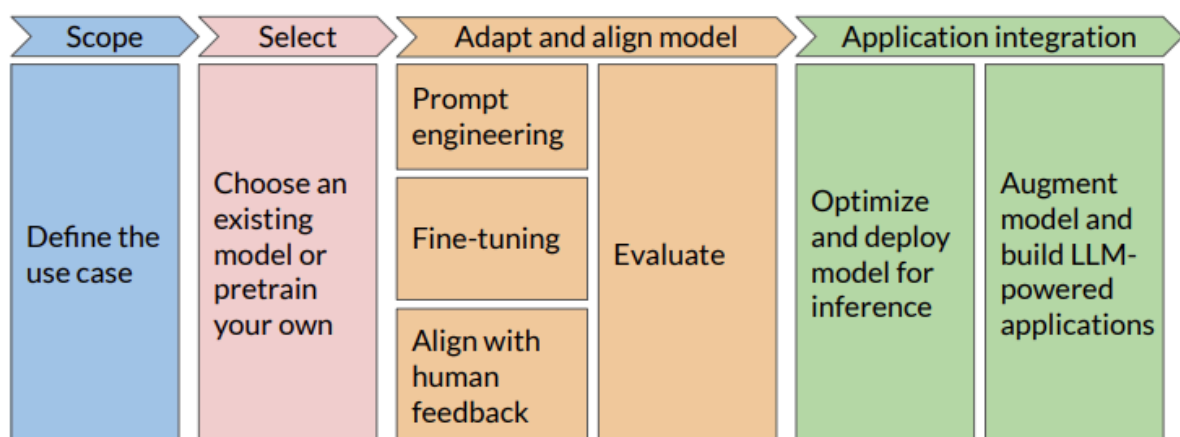
Configuration parameters of the model that can influence the final decision about the next-word generation. These configurations are invoked at the inference time

1. Max New tokens - To limit the number of tokens that the model will generate
2. Greedy vs random sampling - The softmax output generates probability distribution across the dictionary of words. The selection can be
 - a. Greedy - default. model choses word with highest probability. susceptible to repeated sequences words
 - b. Random sampling - sounds more natural and creative. Model can wander-off not making sense
 - i. Top-K sampling - Select the output from top k results with the highest probability, allow to select relevant and probable words without risk of repetition. More likely to sound reasonable
 - ii. Top-p sampling - select an output at random with the top-ranked consecutive results by probability with a cumulative probability $\leq p$
3. Temperature - Assigns the shape of the distribution that the model calculates for the next token. The higher the temperature, the higher the randomness.

- Is a scaling factor applied to the final soft-max layer that influences the shape of the probability distribution of the next token
- Temperature = 1, leave default soft-max values
- Temperature > 1 (warm) - broader and flatter probability distribution - more creative
- Temperature < 1 (cold) - strongly peaked probability distribution



Generative AI project lifecycle



- Scope: Function for the model - single task or many tasks and compute cost?
- Own model or existing model? - In general starting with existing model

3. Adapt and Align - Model Performance - start with In-context Learning. Sometimes may also require Fine Tuning and Reinforcement Learning with Human Feedback (RLHF)
4. Application integration
 - a. Optimize model for deployment and inference
 - b. Additional infrastructure that may be required

Labs

- Using Amazon Sage maker
-

LLM pre-training and scaling laws

Pre-training large language models

Based on the use-case, you might choose a foundation model or train an LLM from scratch. Generally, using an existing foundation model would work just fine.

Model Cards

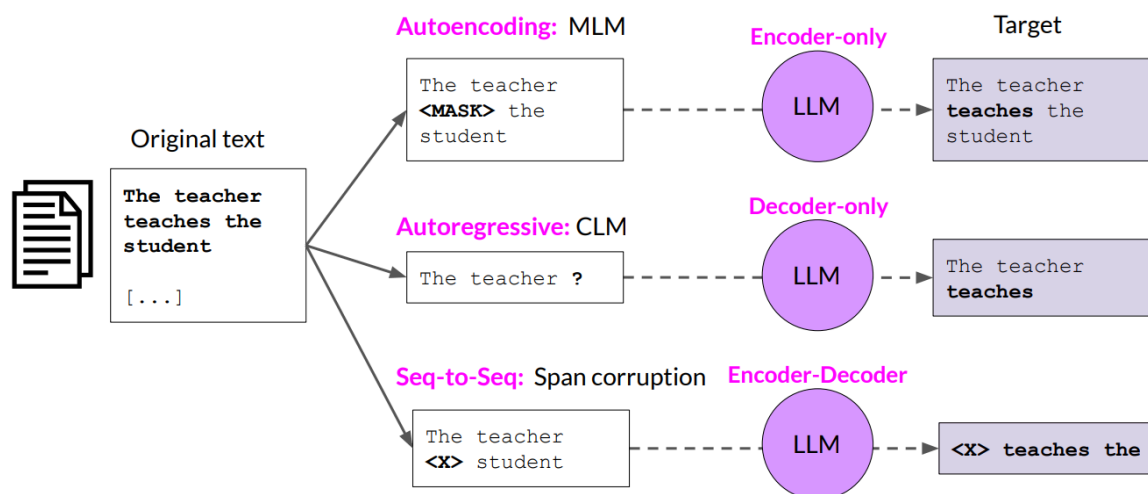
- How the model was trained
- Use cases
- Known limitations of the model

Model architectures and pre-training objectives

- LLMs encode deep statistical representation of language during pre-training phase. During training, the model weights get updated to minimize the loss function of the model.

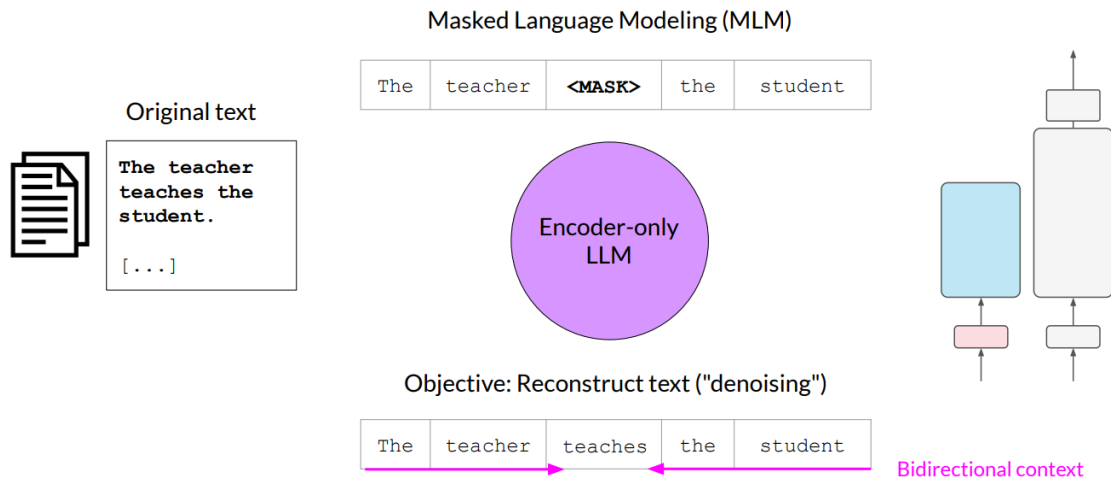
- LLM requires large amount unstructured text for training which could pre-assembled sources of text designed for NLP training or scrapes of text. The scraped text from internet requires cleaning data to improve quality leading to 1-3% of original tokens being used for training.

Three Architectures



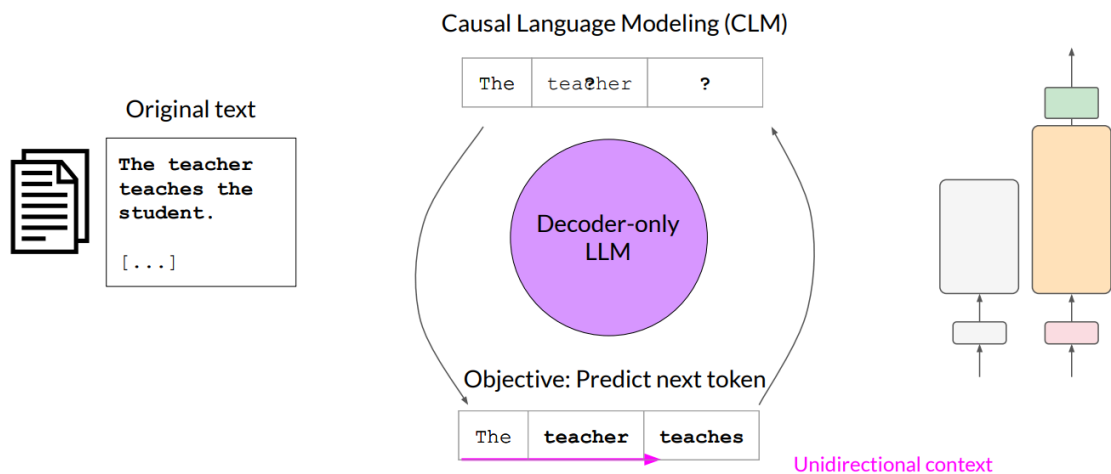
1. Encoder only models - also called autoencoding

- a. trained via **masked language modeling** - masking random text within the input sequence and the LLM is tasked to predict the masked word also called the **denoising** objective
- b. Model learns to develop full context of the token by building bidirectional representations
- c. Use Cases: Sentiment Analysis , Named Entity Recognition, Word Classification
- d. Examples: BERT, ROBERTA



2. Decoder only models - Autoregressive Models

- pre-trained via Causal language Modeling aimed to predict the next token based on the previous sequence of tokens
- Context is unidirectional and model does not know the end of the sequence
- Use Cases - text generation, zero-shot capabilities
- Examples: GPT, BLOOM



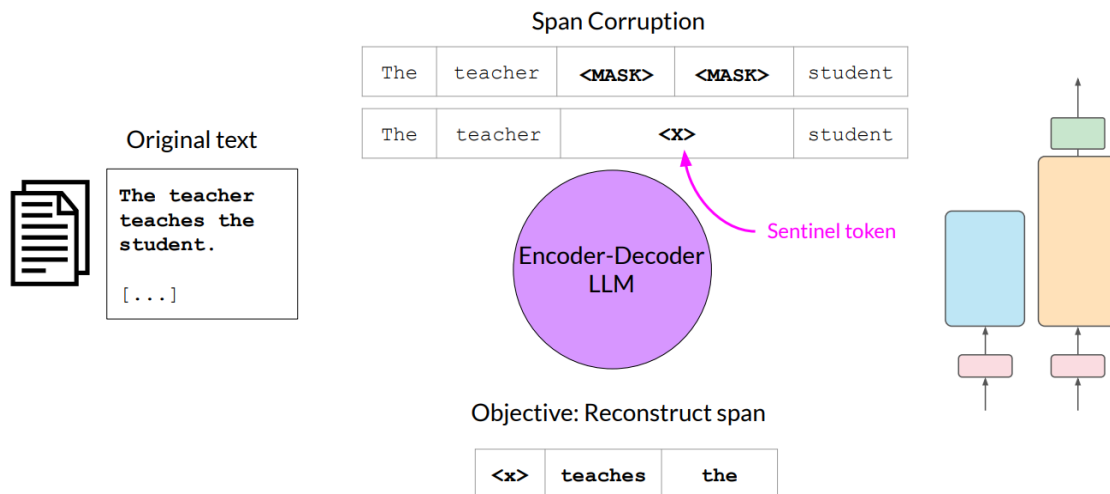
3. Sequence to sequence models

- Uses both encoder and decoder parts of the 🦋 Transformer Architecture
- Pre-training differs from model to model. Popular model like T5 is trained by using Span Corruption which masks random sequences of

input tokens, masked sequences are replaced with a unique **sentinel token** - **special tokens added to the vocabulary but do not correspond to any actual word on the input text**. The decoder is tasked to reconstructing the masked token sequences auto-regressively. The output is the sentinel token followed by the predicted tokens

c. Use Cases: Translation, Text Summarization, Question Answering

d. Examples: T5, BART



Computation Challenges for pre-training large models

CUDA - Compute Unified device architecture is a collection of libraries and tools developed for NVIDIA GPUs. 🖥️ PyTorch and 🖥️ TensorFlow use CUDA to boost performance of matrix multiplications and common operations required for 🧠 Deep Learning

Scale of the computational challenge

Amount of GPU RAM required to store 1B parameters

- 1 param = 4 bytes (32 bit float)
- 1B parameters = $4 * 10^9$ bytes = 4GB just to store the weights
- During training additional parameters are used like
 - 2 adam optimizer states = 8 bytes/param

- Gradients = 4 bytes/parameter
- Activations and temp memory = 8 bytes/parameter

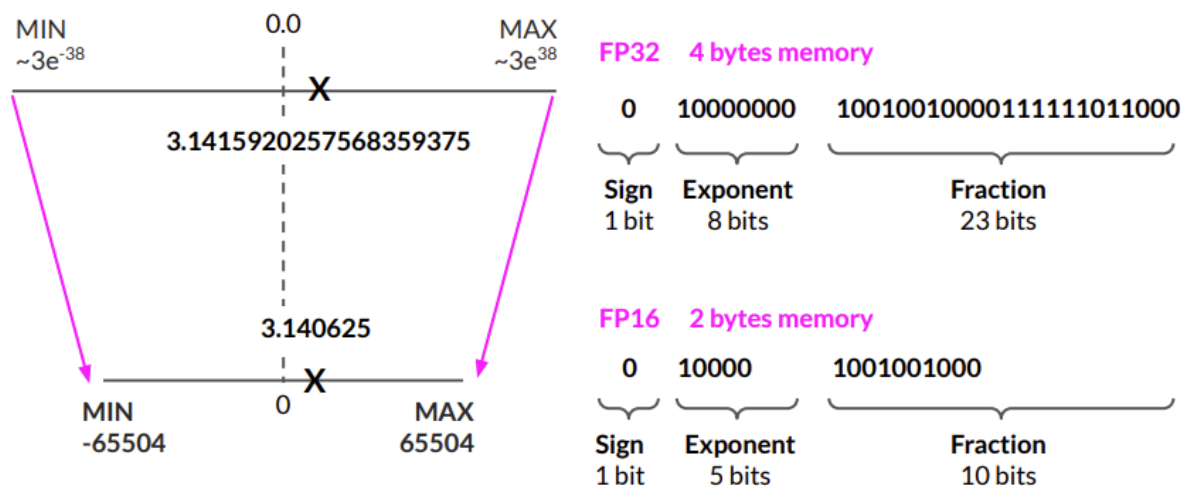
This can lead to 20 extra bytes of memory per parameter during training. In general require 6 times the amount of GPU memory of the memory needed to store the model i.e. 24GB of GPU memory for 1B parameter model. Too high for consumer grade GPUs

Reducing the memory requirement using Quantization

Reduce the memory required to store the weights of the model by reducing the precision from 32bit floating point numbers to 16 point floating or 8 bit integers

- FP32 = Default method = Range of numbers than can be represented $-3e38$ to $+3e38$
- FP16 = Projection of FB32 numbers on to 16 bits using scaling factors

Example: Storing the value of Pi from 32bit float.

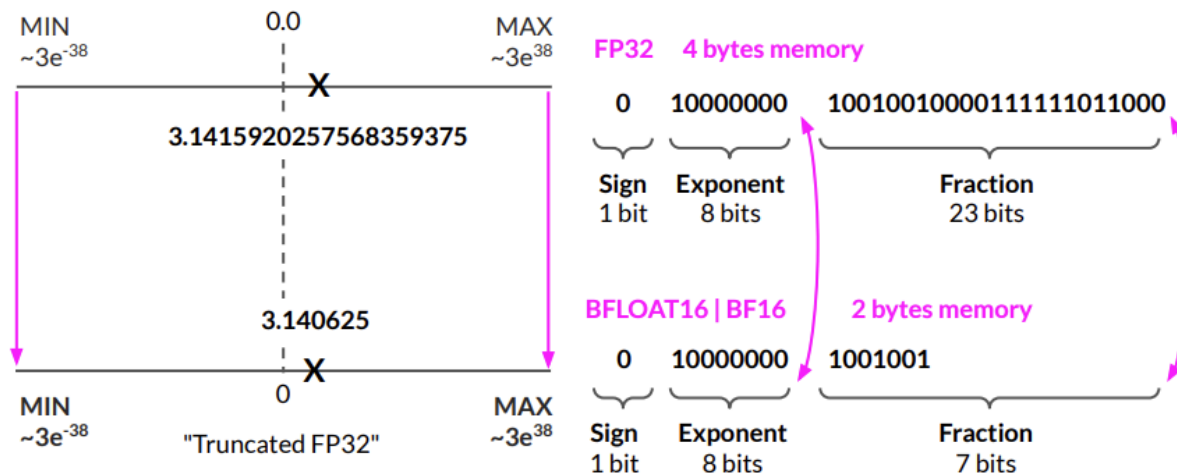


Computers store information in bits of 0 and 1.

- **FP32** uses 1 bit for sign, 8 bits for exponent and 23 bits for fraction (also called Mantissa/Significand) = Precision
- **FP16** = 1 bit for sign + 5 bits for exponent + 10 bits for fraction = Range from -65504 to 65504. Loss of some information with this projection. Loss in precision is acceptable in most cases → Requires on 2 bytes of memory.
Reduces memory requirement by half
- **BFLOAT16** = BRAIN Floating Point 16 bit (developed by Google Brain). Many models are trained with this precision. Hybrid between Float 32 and

Float 16. Fraction bits are truncated from 23 bits to 7 bits → **Leads to training stability & reduces memory requirement by half.**

- Not well suited for integer calculations



- INT8: 1 bit for sign + 7 bits for fraction = Range -128 to 127. Memory requirement by 1/4th = 1 byte

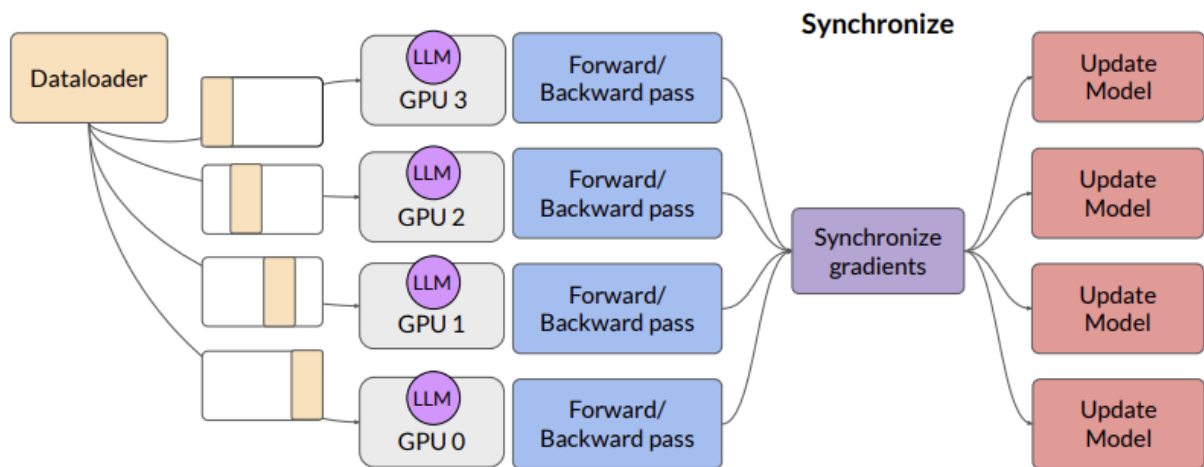
	Bits	Exponent	Fraction	Memory needed to store one value
FP32	32	8	23	4 bytes
FP16	16	5	10	2 bytes
BFLOAT16	16	8	7	2 bytes
INT8	8	-/-	7	1 byte

- Reduce required memory to store and train models
- Projects original 32-bit floating point numbers into lower precision spaces
- Quantization-aware training (QAT) learns the quantization scaling factors during training

Impact: Reduces the memory requirement from 24GB to 12 GB with FP16 and BFLOAT16 and 6GB with INT8. However, most models are now 100s of Billions of parameters, requiring 100s of GPUs. Distributed training is used to split the model across multiple GPUs for training.

Optional : Efficient multi-GPU compute strategies

Distributed Data Parallel



Case: Data fits on single GPU, but need performance boost with parallel training

 [PyTorch](#) has Distributed Data Parallel (DDP) → Copies model into each GPU and sends batches of data in parallel → each dataset is processed in parallel → synchronization step combines the results of each GPU → updates the model on each GPU with an identical copy

- Requires the weights, optimizer states and gradient parameters fit on a single GPU

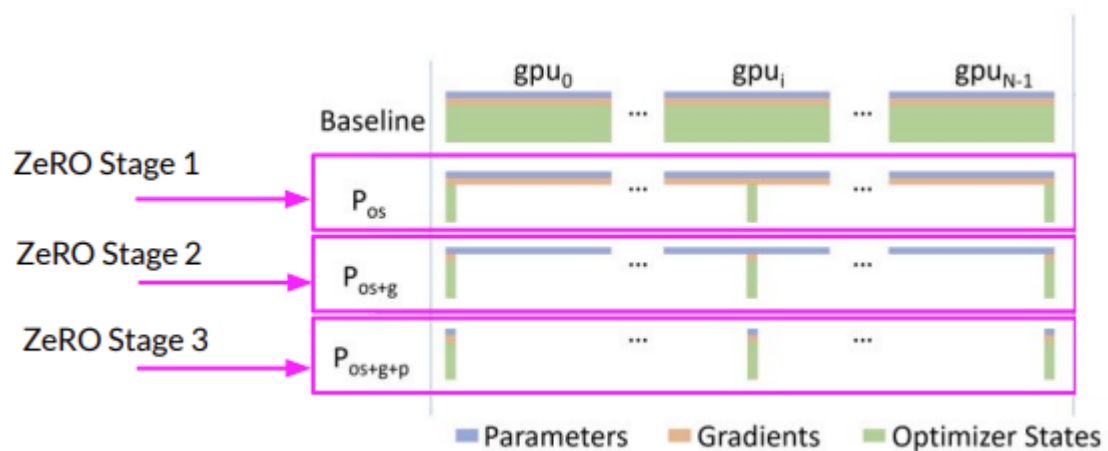
Model Sharding

If the model is too big to fit on a single GPU → use model sharding.  [PyTorch](#) has implementation for **Fully Sharded Data Parallel (FSDP)**

- Motivated by ZeRO paper published by Microsoft Researchers in 2019. ZeRO stands for Zero Redundancy Optimizer. The objective of ZeRO is optimize GPU memory by sharding or distributing model states across GPUs with Zero data overlap.

ZeRO

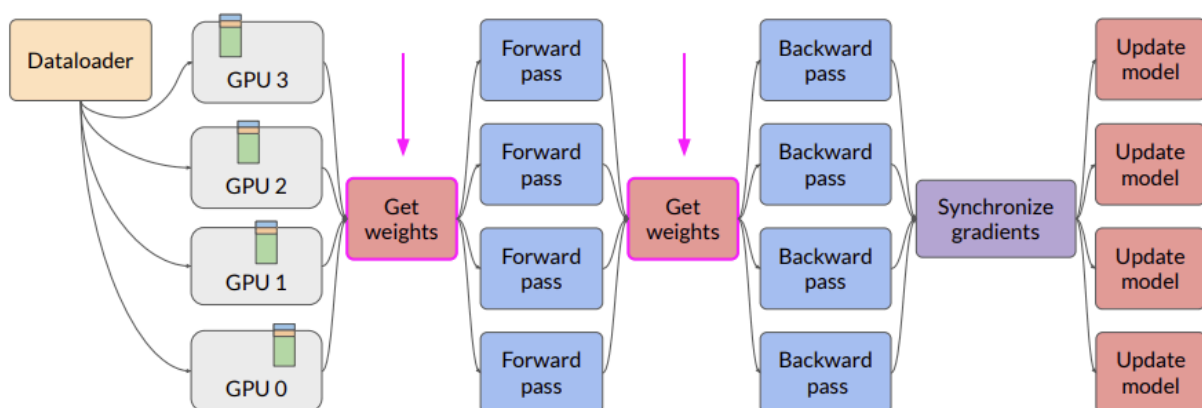
- The amount of memory used in DDP is redundant because the model weights, optimizer states and gradients are all stored redundantly on every GPU.
- ZeRO reduces the redundancy in 3 stages



- Stage 1: Sharding optimizer states across GPUs while keeping copies of model weights and gradients (optimizer stages consume 2x memory than model weights)
- Stage 2: Sharding both optimizer states and gradients across GPUs while keeping copies of model weights
- Stage 3: Sharding all of optimizer states, model weights and gradients. The factor becomes linear with the number of GPUs

FSDP

Distribute weights, gradients and optimizer states across GPUs. However, it requires you to collect the weights across GPUs for forward and backward pass. This requires more GPU RAM because the GPU requests weights from every other GPU on demand and can be stored for future operations. The synchronize step is same as DDP.

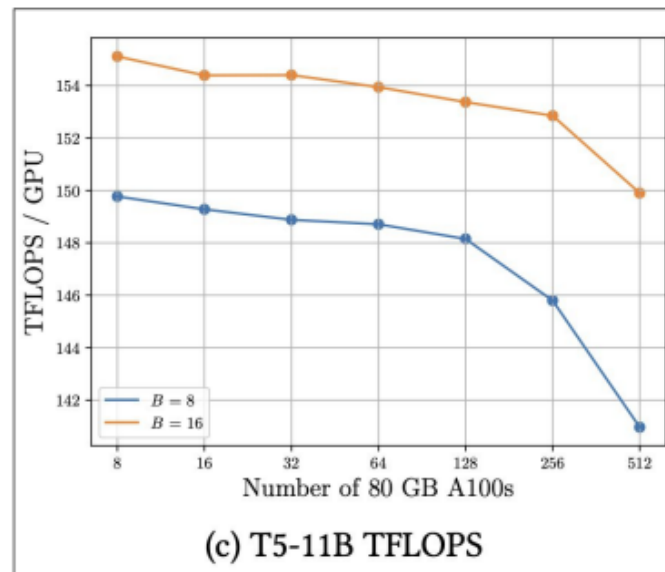


FSDP helps reduce overall GPU memory utilization

Sharding Factor

- No Sharding (1GPU)
- All GPU - Full Sharding
- Hybrid sharding

Higher GPUs are not better, communication between GPUs causes performance to decrease by 7% as shown in chart below



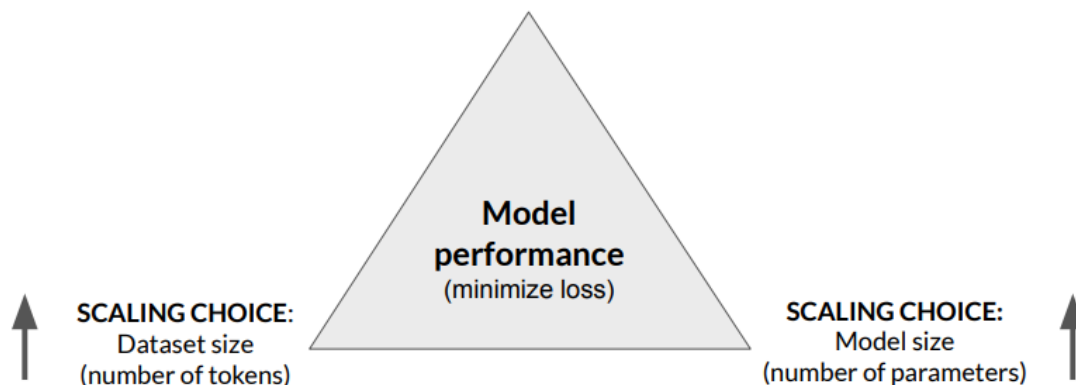
Scaling laws and compute-optimal models

A relationship between model size, training, configuration and performance exists

- **Goal:** Maximize model performance = Minimize loss by either increasing the **dataset size** (number of tokens) and the **model size** (number of parameters) with a constraint that there exists a **compute budget** (GPUs, training time, cost)

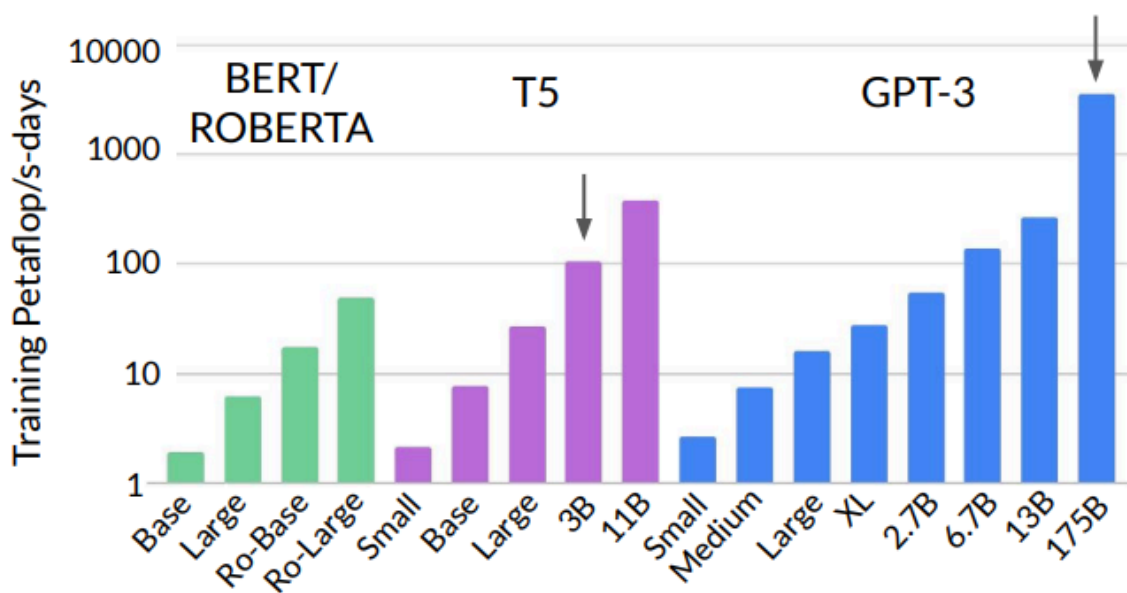
Goal: maximize model performance

CONSTRAINT:
Compute budget
(GPUs, training time, cost)



Limit of Compute = 1 petaflops/s-day = number of floating point operations performed at the rate of 1 peteFLOP per second for one day

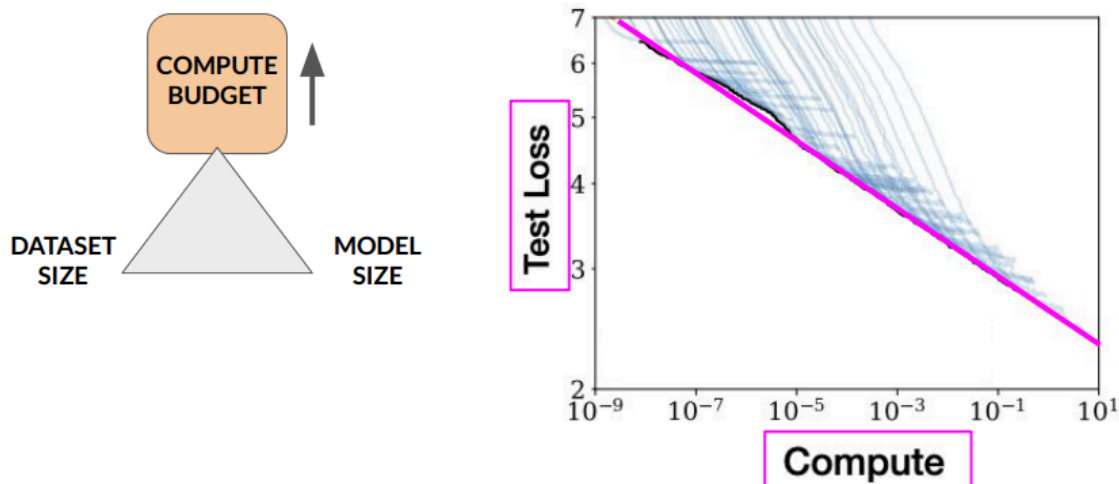
- 1petaFLOP = 1,000,000,000,000,000 (quadrillion) floating point operations/sec = 8 NVIDIA V100 GPUs = 2 NVIDIA A100s
- Compute budget required for pre-training various models



et al. 2020, "Language Models are Few-Shot Learners"

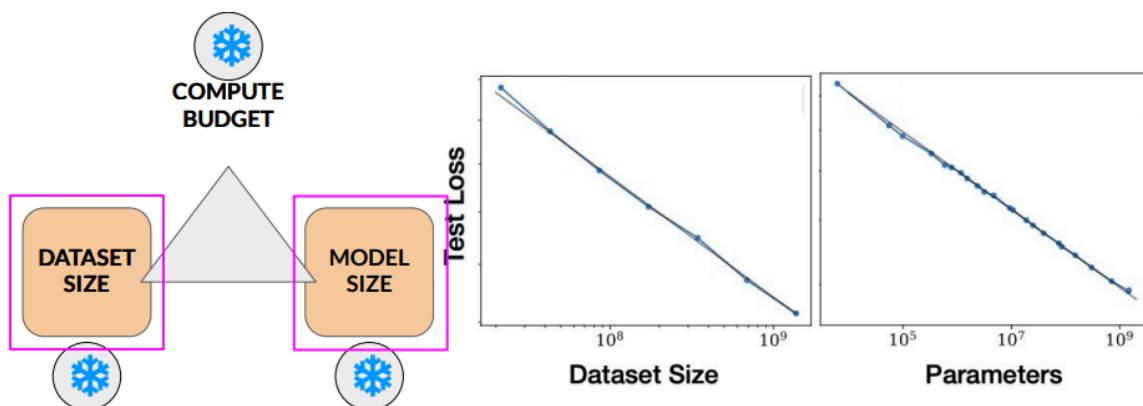
Researchers have found out trade off between dataset size - model size - compute budget

- Higher Compute budget leads to better trained models and can be described by power law relationship



Source: Kaplan et al. 2020, "Scaling Laws for Neural Language Models"

- Compute budget usually constrained by the hardware, project timeline and financial budget. The two levers dataset size - model size also describes power law relationship. As the model increases in size, the performance increases



The Ideal size? - Chinchilla

Optimal number of parameters for a given compute budget and training data. Very large models are **over-parameterized and under-trained**. Smaller models trained on more data could perform as well as large models

The optimal training size is 20x the number of parameters. In the table below, GPT-3 is a large model but is under-trained with only 300B tokens vs the required tokens being 3.5T. LLaMA on the other hand is optimally trained where the actual tokens are close to Chinchilla number.

Model	# of parameters	Compute-optimal* # of tokens (~20x)	Actual # tokens
Chinchilla	70B	~1.4T	1.4T
LLaMA-65B	65B	~1.3T	1.4T
GPT-3	175B	~3.5T	300B
OPT-175B	175B	~3.5T	180B
BLOOM	176B	~3.5T	350B

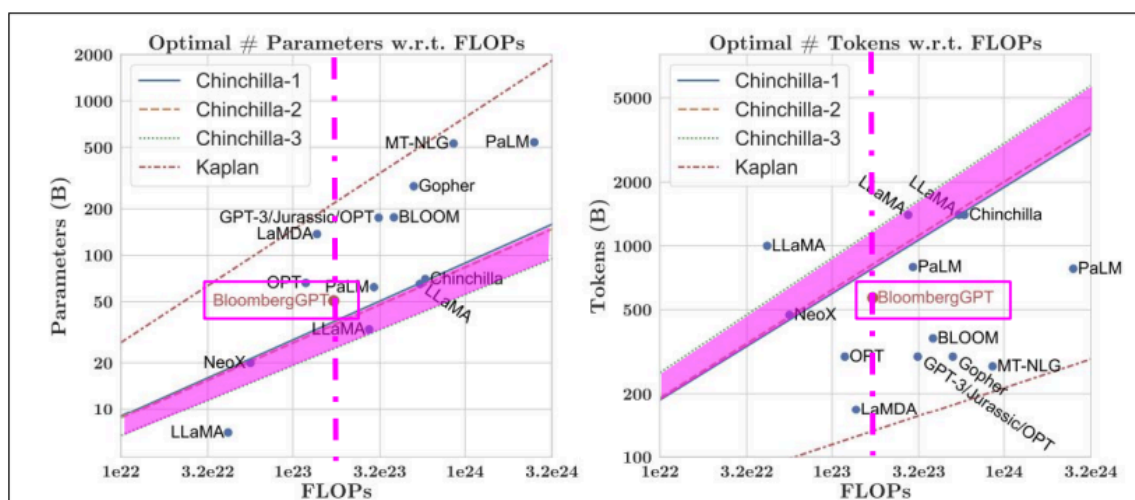
Compute optimal training datasize
is ~20x number of parameters

Compute optimal chinchilla models outperforms non-compute-optimal models such as GPT-3 on large range of downstream evaluation tasks.

The models may not be bigger but more optimal. Smaller models can be expected to outperform than large models with the Chinchilla paper in mind.

Pre-training for domain adaptation

- Existing LLMs may not be suited for domains like Legal (where certain vocabulary is rare and used in different context than generally understood) and medicine (usually has a lot of short forms and uncommon words)
- BloombergGPT was trained for finance adaptation - maintains good performance on both general purpose and finance related tasks
 - Trained on 51% financial data and 49% public data
 - Use Chinchilla scaling laws for guidance



Source: Wu et al. 2023, "BloombergGPT: A Large Language Model for Finance"

Domain-specific training: BloombergGPT



BloombergGPT, developed by Bloomberg, is a large Decoder-only language model. It underwent pre-training using an extensive financial dataset comprising news articles, reports, and market data, to increase its understanding of finance and enabling it to generate finance-related natural language text. The datasets are shown in the image above.

During the training of BloombergGPT, the authors used the Chinchilla Scaling Laws to guide the number of parameters in the model and the volume of training data, measured in tokens. The recommendations of Chinchilla are represented by the lines Chinchilla-1, Chinchilla-2 and Chinchilla-3 in the image, and we can see that BloombergGPT is close to it.

While the recommended configuration for the team's available training compute budget was 50 billion parameters and 1.4 trillion tokens, acquiring 1.4 trillion tokens of training data in the finance domain proved challenging. Consequently, they constructed a dataset containing just 700 billion tokens, less than the compute-optimal value. **Furthermore, due to early stopping, the training process terminated after processing 569 billion tokens.**

The BloombergGPT project is a good illustration of pre-training a model for increased domain-specificity, and the challenges that may force trade-offs against compute-optimal model and training configurations.

You can read the BloombergGPT article [here](#).

Week 1 resources

Below you'll find links to the research papers discussed in this weeks videos. You don't need to understand all the technical details discussed in these papers - **you have already seen the most important points you'll need to answer the quizzes** in the lecture videos.

However, if you'd like to take a closer look at the original research, you can read the papers and articles via the links below.

Generative AI Lifecycle

- **Generative AI on AWS: Building Context-Aware, Multimodal Reasoning Applications** - This O'Reilly book dives deep into all phases of the generative AI lifecycle including model selection, fine-tuning, adapting, evaluation, deployment, and runtime optimizations.

Transformer Architecture

- **Attention is All You Need** - This paper introduced the Transformer architecture, with the core "self-attention" mechanism. This article was the foundation for LLMs.
- **BLOOM: BigScience 176B Model** - BLOOM is a open-source LLM with 176B parameters trained in an open and transparent way. In this paper, the authors present a detailed discussion of the dataset and process used to train the model. You can also see a high-level overview of the model [here](#).
- **Vector Space Models** - Series of lessons from DeepLearning.AI's Natural Language Processing specialization discussing the basics of vector space models and their use in language modeling.

Pre-training and scaling laws

- **Scaling Laws for Neural Language Models** empirical study by researchers at OpenAI exploring the scaling laws for large language models.

Model architectures and pre-training objectives

- **What Language Model Architecture and Pretraining Objective Work Best for Zero-Shot Generalization?** - The paper examines modeling choices in large pre-trained language models and identifies the optimal approach for zero-shot generalization.
- **HuggingFace Tasks and Model Hub** - Collection of resources to tackle varying machine learning tasks using the HuggingFace library.
- **LLaMA: Open and Efficient Foundation Language Models** - Article from Meta AI proposing Efficient LLMs (their model with 13B parameters outperform GPT3 with 175B parameters on most benchmarks)

Scaling laws and compute-optimal models

- **Language Models are Few-Shot Learners** This paper investigates the potential of few-shot learning in Large Language Models.
- **Training Compute-Optimal Large Language Models** Study from DeepMind to evaluate the optimal model size and number of tokens for training LLMs. Also known as "Chinchilla Paper".
- **BloombergGPT: A Large Language Model for Finance** - LLM trained specifically for the finance domain, a good example that tried to follow chinchilla laws.